

CODE SPORT



Sandya Mannarswamy

In this month's column, let us continue our discussion on dynamic languages, focusing on JavaScript.

In last month's column, we explored dynamic languages and how they are translated. While Web 2.0 accelerated the development of many dynamic languages, the one that has come to rule them all as the most popular on the Internet is JavaScript. In this month's column, I will provide a quick overview of JavaScript, discuss the various translation engines available and elaborate on how they differ under the hood. If you're curious about why I'm focusing on JavaScript, I have had a few readers writing in, requesting the magazine to focus more on Internet technologies in this column. So, over the next few months, topics related to Internet technologies will be covered.

JavaScript and prototype-based programming

JavaScript is ubiquitous in today's Internet world. It runs applications all the way from your pocket smartphone to large high-end servers. It has been dubbed as the assembly language of the Internet. Most websites run JavaScript, which was designed in 1995 by Brendan Eich at Netscape. It was originally aimed at enabling non-programmers to enhance websites using client-side executable code. It is interesting to note that though the names of Java and JavaScript are similar, the two languages do not resemble each other in their behaviour to any great extent. Though JavaScript is an object-oriented language, it does not support classes or encapsulation found in traditional object-oriented languages like Smalltalk, Java, C++, etc. It supports what is known as prototype-based programming. In a language like Java, the structure and behaviour of object instances is defined by the class to which the instances belong. Only the instance data is contained in the object itself. Inheritance is supported by having a derived class inheriting structure and behaviour from a base class. In other words, the members and methods of the base class are inherited by (and will be available to) the derived class. This is known as class-based

inheritance. Consider the following code snippet:

```
class Shape{
    void draw();
};
class Circle{
    ...
    Circle c1;
    c1.draw();
}
```

Objects of class Circle inherit the method 'Draw' from the base class Shape. This is class-based inheritance as we know in C++, Java, etc. JavaScript does not support classes. In that case, how can you support behaviour reuse? Well, JavaScript supports behaviour reuse by what is known as prototype-based programming. While you can obtain behaviour reuse in C++ or Java by creating an instance of a class, in JavaScript, it is done by cloning an existing object—which is known as the prototype. So how does inheritance work in prototype-based programming? Consider the following example of a vehicle object:

```
var vehicle =
{ wheels: 4,
  year:"unknown",
  make:"unknown"
};
```

An object in JavaScript is a set of properties, a mutable map from strings to values. The 'vehicle' object above has three properties: 'wheels', 'year', 'make'. Each object has a prototype field, which refers to another object. The prototype object acts as the parent for the current object. Now, you can create a new 'car' object by cloning the 'vehicle' object. (Note that there is no 'clone' operator in JavaScript, but you can write your own clone function.)